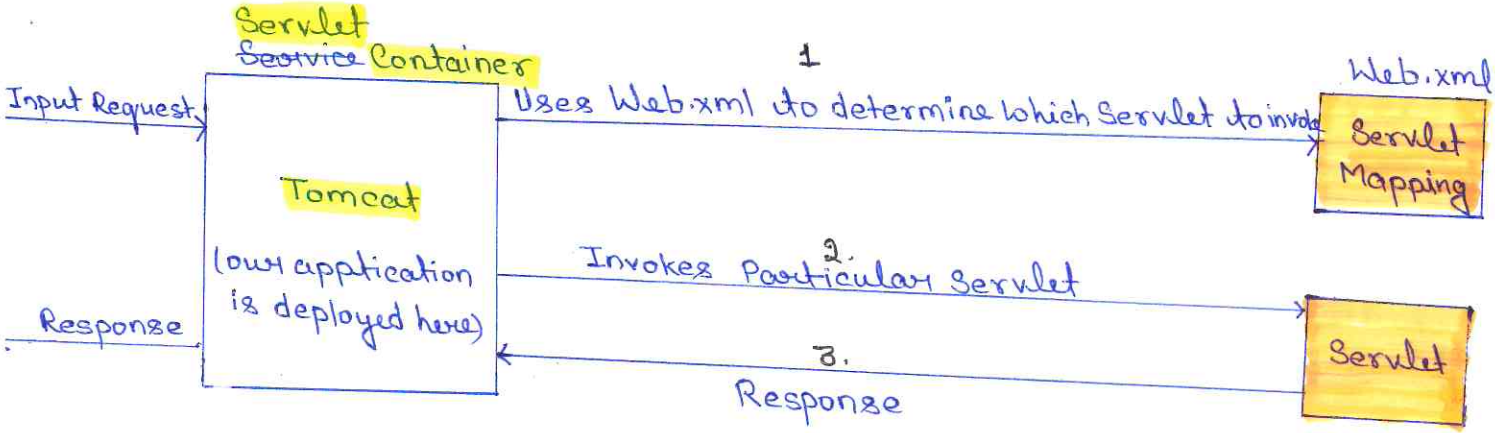
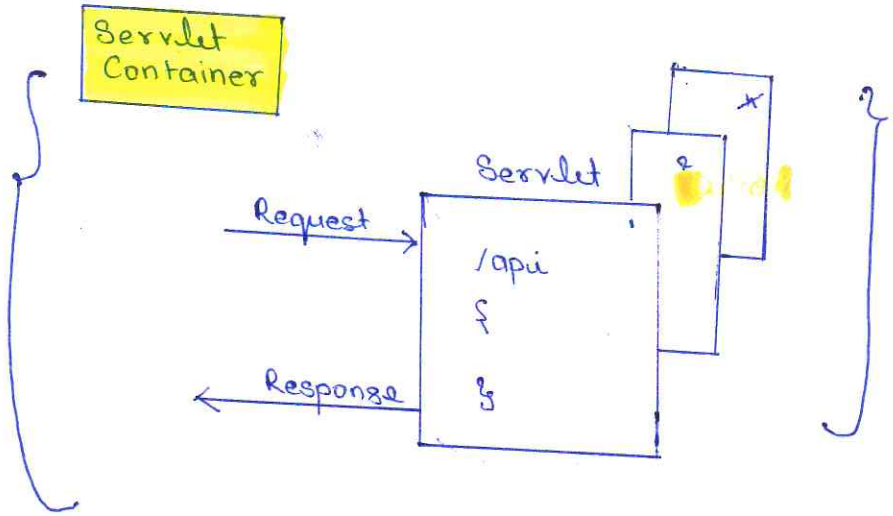
Spring boot : Introduction

Before we talk about Spring or SpringBoot, first we need to understand about "SERVLET" and "SERVLET CONTAINER".

- Provide foundation for building web applications.
- Servlet is a Java Class, which handles client request, process it and return the response.
- And Servlet Container are the ones which manages the Servlet.





* Servlet Container manages the servlet.

Concept && Coding

Servlet 1:

```
@WebServlet("/demoservletone/*")
Public class DemoServlet1 extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) {

        String requestPathInfo = request.getPathInfo();
        if(requestPathInfo.equals("/")){
            // do something
        }
        else if(requestPathInfo.equals("/firstendpoint")) {
            // do something
        }
        else if (requestPathInfo.equals("/secondendpoint")) {
            // do something
        }
    }

    @Override
    protected void doPut(HttpServletRequest request,
                         HttpServletResponse response){
        // do something
    }
}
```

Concept && Coding

Web.xml

```
<servlet>
    <servlet-name> DemoServlet1 </servlet-name>
    <servlet-class> DemoServlet1 </servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name> DemoServlet1 </servlet-name>
    <url-pattern> /demoservletone </url-pattern>
    <url-pattern> /demoservletone/firstendpoint </url-pattern>
    <url-pattern> /demoservletone/secondendpoint </url-pattern>
</servlet-mapping>


<servlet>
    <servlet-name> DemoServlet2 </servlet-name>
    <servlet-class> DemoServlet2 </servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name> DemoServlet2 </servlet-name>
    <url-pattern> /demoservlettwo </url-pattern>
</servlet-mapping>
```

Servlet 2:

```
@WebServlet("/demoservlettwo/*")
public class DemoServlet2 extends HttpServlet{

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response){

        //do something
    }

    @Override
    protected void doPut(HttpServletRequest request,
                         HttpServletResponse response){

        //do something
    }
```

Concept && Coding

Spring Framework, solve challenges which exits with Servlets.

- **Removal of web.xml:**
  1. this web.xml over the time becomes too big and becomes very difficult to manage and understand.
  2. Spring Framework introduced Annotations based Configuration.

- **Inversion of Control (IOC):**
  1. Servlets depends on Servlet container to create objects and maintain its lifecycle.
  2. IoC is more flexible way to manage object dependencies and its lifecycle (through Dependency injection).

- **Unit Testing is much harder:**
  1. As the object creation depends on Servlet container, mocking is not easy. Which makes Unit testing process harder.
  2. Spring dependecy injection facility makes the Unit testing very easy.

- **Difficult to manage REST APIs:**
  1 Handling different HTTP methods, request parameters, path mapping make code little difficult to understand.
  2 Spring MVC provides an organised approach to handle the requests and its easy to build RESTful APIs

There are many other areas where Spring framework makes developer life easy such as: integration with other technology like hibernate, adding security etc....

Concept && Coding

The most important feature of Spring framework is DEPENDENCY INJECTION or Inversion of Control (IoC).

Let see an example without Dependency Injection

```
public class Payment {

    User sender = new User();

    void getSenderDetails(String userID){
        sender.getUserDetails(userID);
    }
}
```

```
public class User {

    public void getUserDetails(String id){
        // do something
    }
}
```

Payment class is creating an instance of User class, and there is one Major problems with this and i.e.

Tight coupling: Now payment class is tightly coupled with User class.

How?

-> Suppose I want to write Unit test cases for Payment "getSenderDetails" method, but now I can not easily MOCK "User" object, as Payment class is creating new object of User, so it will invoke the method of User class too.

Concept && Coding

-> Suppose in future, we have different types of User like "admin", "Member" etc., then with this logic, I can not change the user dynamically.

Now, lets see an example with Dependency Injection.

```
@Component
public class Payment{
  @Autowired
  User sender;

  void getSenderDetails(String userID){
     sender.getUserDetails(userID);
  }
}
```

```
@Component
Public class User{
  Public void getUserDetails
                (String id){
    //do Something
  }
}
```

@Component : tells Spring that, you have to manage this class or bean.

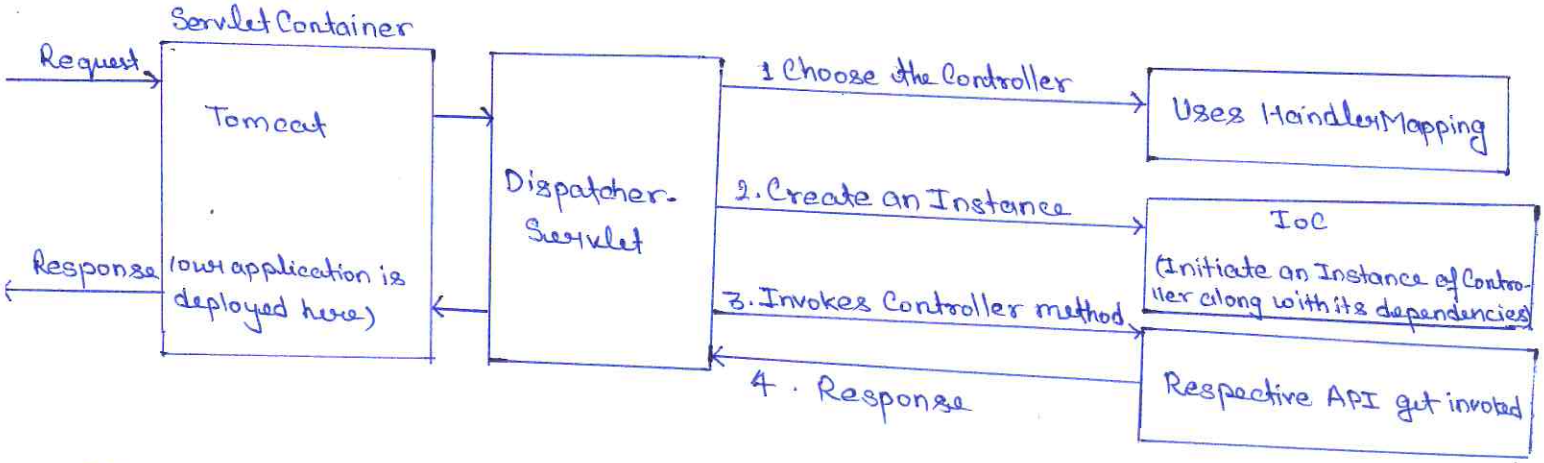
@Autowired : tells Spring to resolve and add this dependency.

The another important feature of Spring framework is lot of **INTEGRATION** available with ~~another fra~~ other frameworks.

This allow Developers to choose different combination of technologies and framework which best fits ~~there~~ their requirement like.

- Integration with Unit testing framework like Junit or Mockito.
- Integration with Data Access framework like Hibernate, JDBC, JPA etc.
- Integration with Asynchronous programming.
- Similar way, it has different integration available for:
  - Caching
  - Messaging
  - Security etc.

Concept && Coding



pom.xml

```
<dependency>
    <groupID> org.springframework</groupId>
    <artifactId> spring-webmvc</artifactId
    <version> 6.1.4</version>
</dependency>
```

①

② servlet-api

③ Junit

Controller class

```
@Controller
@RequestMapping("/paymentapi")
Public class PaymentController{
 @Autowired
 PaymentDAO paymentService;

 @GetMapping("/payment")
 public String getPaymentDetails(){
  return paymentService.getDetails();
}
```

Config class

```
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.conceptandcoding")
Public class AppConfig{
    // add configuration here is required
}
```

Concept && Coding

Dispatcher Servlet class

```
public class MyApplicationInitializer extends AbstractAnnotationConfig-
                                                DispatcherServletInitializer{

    @Override
    protected Class<?>[] getRootConfigClasses(){
        return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses(){
        return new Class[]{AppConfig.class};
    }

    @Override
    protected String[] getServletMappings(){
        return new String[]{"/"};
    }
}
```

Concept && Coding

Spring Boot, solve challenges which exits with Spring MVC.

1. Dependency Management: No need for adding different dependencies separately and also their compatible version headache.

```
<parent>
    <groupId> org.springframework.boot</groupId>
    <artifactId> spring-boot-starter-parent</artifactId>
    <version> 3.2.3</version>
    <relativePath/>
</parent>
```

(1) ← looking lookup parent from repository

(2)
```
<properties>
    <java.version> 17 </java.version>
</properties>
```

(3) spring-boot-starter-web — To create REST APIs & web apps.

(4) spring-boot-starter-test — To write test case

→ You write both dependencies inside the pom.xml file (Maven projects).

Concept && Coding

2. Auto Configuration :- No need for Separately configuring "Dispatcher-Servlet", "AppConfig", "EnableWebMvc", "ComponentScan". Spring boot internally by-default.

```
@SpringBootApplication
public class SpringbootApplication{
    public static void main(String[] args){
        SpringApplication.run(SpringbotApplication.class, args);
    }
}
```

3. Embedded Server :-

In traditional Spring MVC application, we need to build WAR file, which is a packaged file containing your application's classes, JSP pages, configuration files, and dependencies.
Then we need to deploy this WAR file to a servlet container like Tomcat.

→ But in Spring boot, Servlet container is already embedded, we don't have to do all this stuff. Just run the application, that's all.

Concept && Coding

So, What is Spring boot?

- It provides a quick way to create a production ready application.
- It is based on Spring framework.
- It support "Convention over Configuration".
  Use default values for configuration, and if developer don't want to go with convention (the way something is done), they can override it.
- It also help to run an application as quick as possible.

① pom.xml

add dependency

spring-boot-starter-web

It automatically adds:

1. Spring MVC (to create APIs & Controllers)
2. Tomcat server (runs your app automatically)
3. REST API support
4. JSON support using Jackson

Concept && Coding

```
@SpringBootApplication
public class SpringbootApplication{
    public static void main(String[] args){
        SpringApplication.run(SpringbootApplication.class, args);
    }
}
```

```
@RestController
@RequestMapping("/myapi")
public class MyController{

    @GetMapping("/firstapi")
    public String getData(){
        return "Hello from concept and coding";
    }
}
```

localhost:8080/myapi/firstapi

Hello from concept and coding

Concept && Coding